

三链协同系统规划设计

结构

每个页面承担一个清晰任务：

页面	核心作用	必须展示的价值
首页驾驶舱	看总体状态	三链协同总览、核心 KPI（可选）
本体知识图谱	看语义关系	Demand → Vehicle → Module → Supplier → Plant → Warehouse → Customer 链
场景配置与数据生成	切换业务场景	预设 2-3 个典型场景，并生成求解输入
一键求解	调用求解器	展示求解过程、阶段目标、日志、结果文件
结果解释分析	看结果与原因	展示 procurement_plan、production_plan、demand_match、gap_detail，并分析选择

首页

放一条流程图：

需求链 Demand Chain

预测需求 / 客户节点 / 交付日期 / 优先级

↓ 通过 Vehicle、BOM、Take Rate 关联

制造链 Manufacturing Chain

车型 / 模块 / BOM / 产线 / 工厂 / 产能

↓ 通过供应节点、运输弧、提前期关联

供应链 Supply Chain

供应商 / 采购能力 / 运输路径 / 库存 / 到货

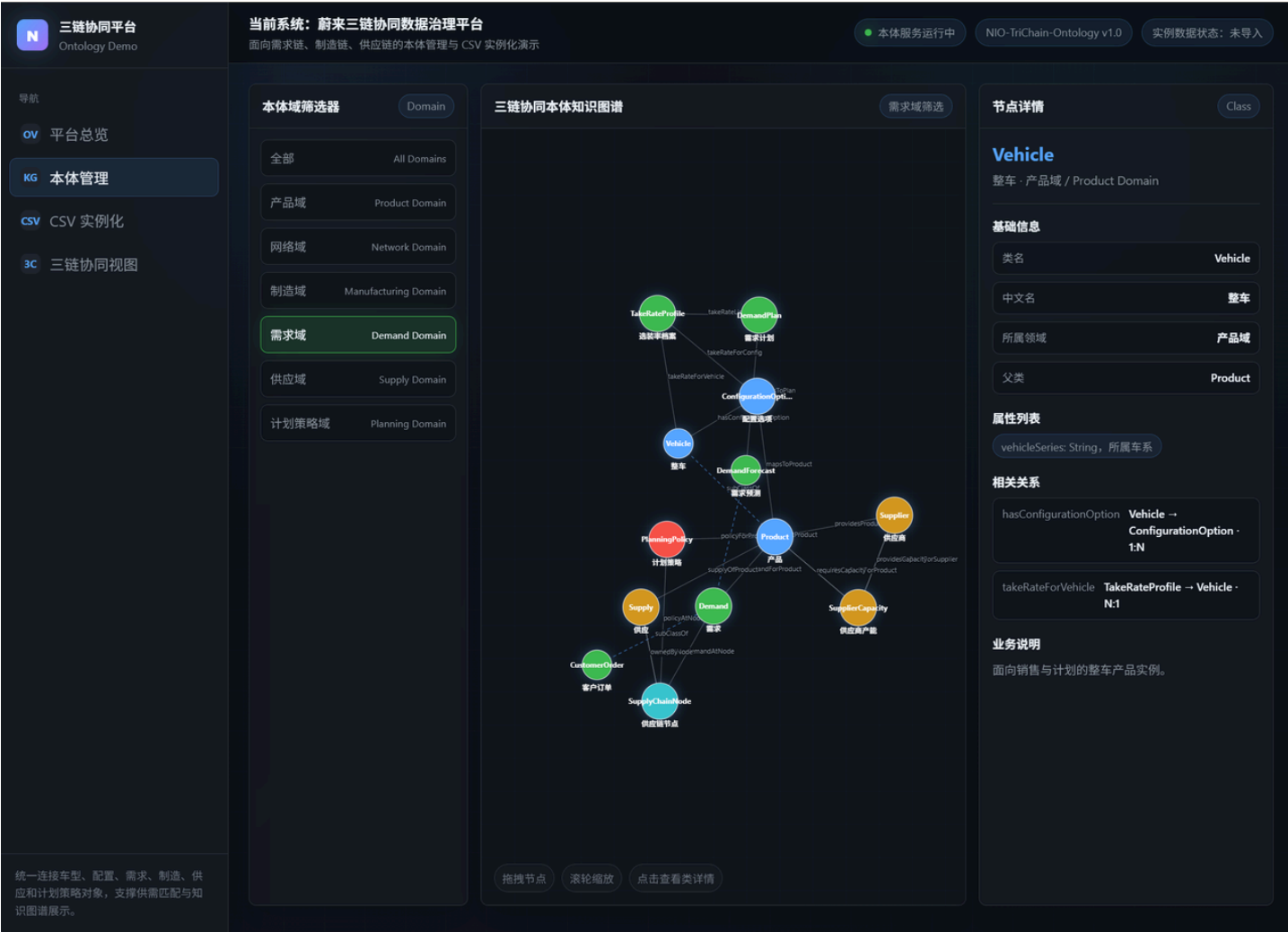
↓

优化求解 Solver

需求满足最大化 → 服务收益最大化 → 成本最小化

模块一：三链协同本体展示

沿用现在的



模块二：业务场景配置与数据仿真

用户可以配置不同业务场景，然后系统基于本体生成对应的实例数据，展示数据生成情况，并决定是否存入数据库，存入形式是末尾添加还是覆盖

在CSV实例化界面上修改

至少包含场景参数、生成数据、生成后的表头和本体类的对应关系、数据校验

该模块面向系统 demo 的“数据准备与实例化”环节，作用是让用户在前端配置一个业务场景，系统根据三链本体和内置仿真规则生成一批完整的实例数据，并将生成结果以 CSV / Excel / MySQL 批次的形式保存，供后续求解器调用。

已经完成后端接口：

Three Chain Simulation Demo API

0.1.0

OAS 3.1

/openapi.json

default



GET	/health	Health	▼
GET	/runs	Get Runs	▼
POST	/runs/generate	Post Generate Run	▼
GET	/runs/{run_id}/tables	Get Tables	▼
GET	/runs/{run_id}/tables/{table_name}	Get Table Rows	▼
GET	/runs/{run_id}/summary	Get Summary	▼
GET	/runs/{run_id}/demand-supply-gap	Get Demand Supply Gap	▼
GET	/runs/{run_id}/capacity-gap	Get Capacity Gap	▼

Schemas



GenerateRunRequest > Expand all object

HTTPValidationError > Expand all object

ValidationError > Expand all object

页面核心任务：

1. 配置业务场景参数；
2. 调用后端数据仿真接口生成实例数据；
3. 展示本次生成的数据批次、数据规模和关键统计；
4. 展示数据表字段和本体类、本体关系之间的对应关系；
5. 对生成数据进行一致性校验；
6. 决定是否写入数据库，以及采用追加写入还是覆盖写入。

2.1 场景参数配置（求解器那边提供需要配置的信息）

————— 目前未实现，因为没有相关知识储备……

用户进入页面后，首先选择或填写本次仿真的场景参数。参数不需要暴露过多底层代码细节，应以业务可理解的方式展示。

1. 场景名称

例如：基础场景、需求高峰场景、供应受限场景、运输延迟场景。

2. 仿真日期

用于确定预测周、订单日期、采购到货日期、在途到货日期等时间字段。

3. 随机种子

用于保证同一组参数下可以复现实验结果。默认使用系统内置 seed，也允许用户手动输入。

4. 业务规模

包括车辆数量、模块数量、供应商数量、工厂数量、仓库数量、客户节点数量、预测需求规模等。

5. 需求侧参数

包括需求波动强度、客户优先级分布、预测需求周期长度、订单需求占比等。

6. 制造侧参数

包括 BOM 复杂度、车辆与模块绑定关系、产线产能水平、工厂可生产车型范围等。

7. 供应侧参数

包括供应商供货能力、采购提前期、在途比例、库存水平、运输路径可达性等。

2.2 数据生成流程

用户点击“生成数据”后，前端调用后端接口：

POST /runs/generate

后端执行流程如下：

1. 读取用户提交的场景参数；
2. 调用数据仿真器生成三链实例数据；
3. 在本地生成一个新的时间戳文件夹，例如：

mock_output_v2/20260512_163732/

4. 将本次生成的 CSV 和 Excel 文件保存到该文件夹；
5. 将同一批数据写入 MySQL，并为所有业务表增加统一的 run_id；
6. 在 simulation_run 表中登记本次运行记录；
7. 向前端返回 run_id、输出目录、生成状态和数据表清单。

run_id 是本次仿真的唯一批次标识。后续查看表数据、摘要统计、供需缺口、产能缺口时，都通过 run_id 查询。

2.3 生成结果概览

数据生成完成后，页面顶部展示本次运行的总体信息。

已实现展示内容：

1. run_id

例如：20260512_163732

2. 运行状态

包括 RUNNING、SUCCESS、FAILED。

3. 输出位置

展示 CSV / Excel 所在文件夹路径。

4. 生成表数量

当前生成 34 张业务数据表。

5. 核心数据规模

包括：

- 车辆型号数量；
- 模块数量；
- 预测需求总量；
- 订单需求总量；
- 物料需求总量；
- 库存数量；
- 在途数量；
- 周产能数量。

这些统计可以通过后端接口获取：

GET /runs/{run_id}/summary

页面可以用指标卡展示，帮助用户快速判断这批数据是否生成成功，以及规模是否符合预期。

6. 生成数据表展示

页面需要提供一个“生成表清单”区域，让用户查看本次生成了哪些表。

后端接口：

GET /runs/{run_id}/tables

表清单建议按业务域分组展示：

1. 主数据与本体映射表

包括 node、vehicle_type、module、configuration_option、bill_of_materials、transportation_lane 等。

2. 需求链数据

包括 demand_forecast、customer_order、service_policy、take_rate_profile_weekly 等。

3. 制造链数据

包括 plant、production_line、production_capacity_week、assembly_process 等。

4. 供应链数据

包括 supplier_capacity_week、purchase_order、supply、inventory_on_hand、fact_inbound_eta 等。

5. 优化与分析友好表

包括 material_need_weekly、dim_lane_reachability、cost_profit_proxy、optimization_ready_set 等。

用户点击某张表后，页面调用：

GET /runs/{run_id}/tables/{table_name}

展示该表的前若干行数据。表格默认分页，避免一次加载过多数据。

7. 表头与本体类的对应关系——可补充

该模块需要明确说明：生成的数据不是孤立字段，而是本体实例化后的结构化数据。每张表的字段都应该能够对应到本体中的类、对象属性或数据属性。

建议在页面中增加“字段映射”视图，展示如下内容：

数据表	字段	对应本体对象	含义
node	node_id	Node	节点实例唯一标识
node	node_type	Node 类型	区分 SUPPLIER、PLANT、WAREHOUSE、CUSTOMER
vehicle_type	vehicle_type_id	VehicleType	车辆型号
module	module_id	Module	模块或关键零部件
bill_of_materials	vehicle_type_id	VehicleType	BOM 所属车型
bill_of_materials	module_id	Module	车型所需模块
bill_of_materials	qty_per_vehicle	consumes	单车模块消耗数量
transportation_lane	from_node_id	Node	运输起点
transportation_lane	to_node_id	Node	运输终点

transportation_lane	lead_time_days	TransportLeadTime	运输提前期
demand_forecast	customer_node_id	Customer	需求客户节点
demand_forecast	vehicle_type_id	VehicleType	需求车型
demand_forecast	forecast_qty	DemandQuantity	预测需求数量
supplier_capacity_week	supplier_node_id	Supplier	供应商节点
supplier_capacity_week	module_id	Module	可供应模块
production_capacity_week	plant_node_id	Plant	工厂节点
inventory_on_hand	owner_node_id	Node	库存所在节点

该映射的作用是为了让用户看到：系统不是随机生成表格，而是在三链本体约束下生成可解释、可追溯的数据实例。

2.4 数据校验

生成数据后，系统需要自动进行一致性校验，并将校验结果展示给用户。———校验部分完成，未展示

1. 主键唯一性校验

例如 vehicle_type_id、module_id、node_id、lane_id 不能重复。

2. 外键引用校验

例如 BOM 中的 vehicle_type_id 必须存在于 vehicle_type 表，module_id 必须存在于 module 表。

3. 节点类型校验

例如供应商到工厂的运输路径必须是 SUPPLIER -> PLANT，工厂到仓库必须是 PLANT -> WAREHOUSE，仓库到客户必须是 WAREHOUSE -> CUSTOMER。

4. 数值合法性校验

需求数量、库存数量、供应能力、运输成本、提前期等字段不能为负数。

5. 时间逻辑校验

采购订单创建日期不能晚于到货日期，在途 ETA 不能早于发运日期。

6. BOM 完整性校验

每个可生产车辆型号必须至少关联一个模块，关键模块不能缺失。

7. 优化输入完整性校验

求解器需要的需求、库存、供应、产能、运输路径、成本收益代理表必须存在且非空。

2.5 数据库存储策略

该模块需要让用户决定生成数据是否写入数据库，以及如何写入。

当前 demo 推荐策略：

1. 每次生成都创建新的 run_id；
2. 每张业务表写入 MySQL 时都增加 run_id 字段；
3. 不直接覆盖历史批次；
4. 前端通过 run_id 选择要查看或求解的数据批次。

这样可以保留多次实验结果，便于比较不同场景。

对于“追加”和“覆盖”的解释：

1. 追加写入

保留历史 run_id，新数据作为新批次写入。适合系统 demo、场景对比和实验复现。

2. 覆盖写入

清空已有业务表或删除某个 run_id 后重新写入。适合只保留最新版数据的演示场景，但不利于回溯。

当前建议默认使用“追加写入”，并在页面上提供“删除指定批次”或“仅查看最新批次”的能力，而不是直接覆盖全部数据。

模块三：求解器调用和结果展示界面（具体展示内容需求解小组核对提供）

主要功能包括：

1. 点击调用求解器接口；
2. 展示求解状态；
3. 展示求解过程日志；
4. 展示求解后生成的结果文件；
5. 可视化分析（展示与哪些数据相关是我们做，其他他们做）

4.1 求解过程展示

1. 三阶段求解卡片

阶段 1：优先保交付

目标：
最大化加权需求满足量。

业务解释：
优先满足高优先级需求，例如 CRITICAL、HIGH 需求。

阶段 2：提升服务收益

目标：
在交付满足基础上，最大化服务收益或业务价值。

业务解释：
在都能满足的情况下，优先满足价值更高、服务收益更高的需求。

阶段 3：降低总成本

目标：
在前两阶段不明显受损的前提下，最小化采购、生产、运输和库存成本。

业务解释：
先保证交付和服务，再在可行范围内降本。

三张卡片的标题、目标和业务解释为静态内容，求解状态和输出内容为动态显示
动态展示这些内容（求解接口需要返回内容）：

动态内容	说明
求解状态	未开始 / 求解中 / 已完成 / 失败
阶段目标值	每一阶段求出来的目标值
阶段耗时	每个阶段求解耗时
约束锁定状态	阶段 2 是否锁定阶段 1 结果，阶段 3 是否锁定阶段 1 和 2
业务结果摘要	满足量、服务分、总成本等
异常提示	如果 infeasible 或数据缺失，显示原因

样例：

求解前

三张卡片显示为“待运行”。

代码块

- 1 阶段 1：优先保交付
- 2 状态：待求解
- 3 目标值：--
- 4 结果摘要：--

代码块

- 1 阶段 2：提升服务价值
- 2 状态：待求解
- 3 目标值：--
- 4 结果摘要：--

代码块

- 1 阶段 3：降低总成本
- 2 状态：待求解
- 3 目标值：--
- 4 结果摘要：--

求解中

当前正在求解的阶段高亮。

代码块

- 1 阶段 1：优先保交付
- 2 状态：求解中
- 3 目标：最大化加权需求满足量

阶段 1 完成后：

代码块

- 1 阶段 1：优先保交付
- 2 状态：已完成
- 3 目标值： $F1 = 320$
- 4 满足量： $320 / 360$
- 5 加权满足率：92.4%

然后阶段 2 变成求解中：

代码块

- 1 阶段 2：提升服务价值

- 2 状态：求解中
- 3 说明：已锁定阶段 1 满足结果

求解完成后

三张卡片都显示结果。

阶段 1 卡片

代码块

- 1 阶段 1：优先保交付
- 2 状态：已完成
- 3
- 4 目标值： $F1 = 320$
- 5 需求满足量： $320 / 360$
- 6 高优先级需求满足率：100%
- 7
- 8 业务解释：
- 9 资源有限时，系统优先保障高优先级需求。

阶段 2 卡片

代码块

- 1 阶段 2：提升服务价值
- 2 状态：已完成
- 3
- 4 目标值： $F2 = 765000$
- 5 服务收益得分：765000
- 6 未满足惩罚：12000
- 7
- 8 业务解释：
- 9 在阶段 1 满足结果基本不下降的前提下，系统选择服务价值更高的需求匹配方案。

阶段 3 卡片

代码块

- 1 阶段 3：降低总成本
- 2 状态：已完成
- 3
- 4 目标值： $F3 = 428000$
- 5 采购成本：260000
- 6 生产成本：110000
- 7 运输成本：38000

8 库存成本：20000
9
10 业务解释：
11 在交付和服务价值基本不受损的前提下，系统选择总成本更低的采购、生产和运输方案。

2. 求解运行日志

展示：

- 数据读取成功
- 本体映射成功
- 中间表生成成功
- 可行变量数量
- 约束数量
- 第一阶段目标值
- 第二阶段目标值
- 第三阶段目标值
- 求解状态
- 求解耗时

3. 可视化求解状态

可以用进度条：

1. 已读取需求数据
2. 已生成 BOM 期望消耗
3. 已识别供应商能力
4. 已构造生产能力约束
5. 已构造运输可达关系
6. 第一阶段求解完成
7. 第二阶段求解完成
8. 第三阶段求解完成
9. 结果文件已生成

4.2 结果展示与可解释分析

展示求解结果，并解释结果背后的业务原因。

页面包括如下五个部分：

- 1. 总体结果概览
- 2. 需求匹配结果
- 3. 采购计划与供应商选择解释
- 4. 生产计划
- 5. 缺口诊断

4.2.1 总体结果概览

功能

让用户先快速知道这次求解结果好不好。

展示内容

只展示 6 个核心指标：

指标	说明
总需求量	当前场景所有需求数量
已满足需求量	成功满足的需求数量
需求满足率	已满足 / 总需求
总缺口量	未满足的需求数量
总采购量	模块采购总量
总生产量	整车生产总量

解释内容

自动生成一句总结即可：

代码块

- ```
1 本次求解总需求量为 500，已满足 430，满足率为 86%，剩余缺口 70。
2 主要缺口集中在 VEH0001，原因是关键模块 MOD0001 供应不足。
```

### 4.2.2 需求匹配结果

## 功能

说明每条需求是否被满足，以及由哪个仓库/路径满足。

## 展示内容

表格只保留核心字段：

| 字段                | 说明    |
|-------------------|-------|
| demand_id         | 需求 ID |
| vehicle_id        | 车型    |
| customer_node_id  | 客户节点  |
| demand_qty        | 需求数量  |
| fulfilled_qty     | 已满足数量 |
| unmet_qty         | 未满足数量 |
| warehouse_node_id | 满足仓库  |
| priority          | 优先级   |

## 点击需求后的解释

点击一条需求，展示一条本体链路：

代码块

1 Demand → Vehicle → Module → Supplier → Plant → Warehouse → Customer

示例：

代码块

1 需求 FDW0001 需要车型 VEH0001。  
2 VEH0001 需要关键模块 MOD0001。  
3 MOD0001 由供应商 NOD0014 供应到工厂 NOD0001。  
4 工厂生产整车后，经仓库 NOD0201 交付给客户节点 NOD0101。

## 4.2.3 采购计划

## 功能

说明系统采购了什么、从谁那里采购，以及为什么选这个供应商。

## 展示内容

表格只保留核心字段：

| 字段                    | 说明     |
|-----------------------|--------|
| module_id             | 模块     |
| supplier_node_id      | 供应商    |
| receiving_node_id     | 接收工厂   |
| business_week         | 采购周    |
| quantity              | 采购数量   |
| unit_procurement_cost | 单位采购成本 |
| lead_time_days        | 采购提前期  |

必须做的解释 1：为什么选更贵但更快的供应商

示例：

代码块

1

模块 MOD0001 有两个候选供应商：

2

3

NOD0011：采购成本低，但提前期 5 天，无法满足需求交付日期；

4

NOD0014：采购成本较高，但提前期 1 天，可以按期到达。

5

6

因此系统选择 NOD0014。

必须做的解释 2：为什么发生多供应商拆分

示例：

代码块

1

MOD0001 当前周需要 150 件。

2

3

NOD0011 最大供应能力为 100 件；

4

NOD0014 最大供应能力为 80 件。

5

6

单一供应商无法满足全部需求，因此系统拆分采购：

7

NOD0011 采购 100 件，NOD0014 采购 50 件。

4.2.4 生产计划

功能

说明系统安排哪个工厂、哪条产线生产什么车型。

展示内容

表格只保留核心字段：

| 字段                 | 说明    |
|--------------------|-------|
| business_week      | 生产周   |
| plant_node_id      | 工厂    |
| production_line_id | 产线    |
| vehicle_id         | 车型    |
| production_qty     | 生产数量  |
| capacity_qty       | 产能    |
| utilization_rate   | 产能利用率 |

简单解释

点击一条生产计划，展示：

代码块

```
1 系统安排 NOD0001 工厂的 LINE_V_001 生产 VEH0001。
2
3 原因：
4 VEH0001 当前周存在需求；
5 LINE_V_001 具备生产 VEH0001 的能力；
6 该产线本周产能充足；
7 所需模块 MOD0001 已通过采购补齐。
```

4.2.5 缺口诊断

功能

说明哪些需求没有完全满足，以及主要原因是什么。

展示内容

表格只保留核心字段：

| 字段                | 说明    |
|-------------------|-------|
| demand_id         | 需求 ID |
| vehicle_id        | 车型    |
| demand_qty        | 需求量   |
| fulfilled_qty     | 已满足量  |
| gap_qty           | 缺口量   |
| gap_reason        | 缺口原因  |
| bottleneck_object | 瓶颈对象  |
| suggestion        | 建议动作  |

## 支持 3 类缺口原因

### 1. 模块供应不足

代码块

- 1 需求 FDW0003 需要 VEH0001。
- 2 VEH0001 需要关键模块 MOD0001。
- 3 当前周 MOD0001 可用量不足，导致 VEH0001 无法完成全部装配。
- 4 因此该需求产生缺口。

### 2. 产线能力不足

代码块

- 1 需求 FDW0004 需要 VEH0002。
- 2 VEH0002 只能在 LINE\_V\_002 生产。
- 3 但 LINE\_V\_002 本周产能不足，无法生产足够整车。
- 4 因此该需求产生缺口。

### 3. 运输或提前期不满足

代码块

- 1 仓库 NOD0202 虽然有车辆，但到客户节点 NOD0101 的运输提前期过长，
- 2 无法满足需求交付日期。
- 3 因此该仓库不能用于该需求履约，最终形成缺口。

解释机制分为三层：

第一层：本体语义层，定义对象是谁、属于什么类型、和谁有关

第二层：优化结果层，告诉系统实际发生了什么

第三层：解释规则层，把本体关系和求解结果组合成业务解释

# 总结

## 本体价值

本体将需求、产品、BOM、产线、供应商、库存、运输和策略统一到同一语义框架中，使三链数据从“字段级集成”升级为“业务对象级协同”。

## 数据映射

系统基于本体自动识别业务对象及其关系，将原始业务数据转换为优化模型可直接读取的中间表，降低人工建模和字段硬编码成本。

## 可解释性

每一条采购、生产、运输和需求匹配结果，都可以回溯到本体中的业务对象和关系，实现从优化结果到业务原因的解释闭环。

## 扩展性

当新增车型、模块、供应商、工厂、替代料或计划策略时，只需扩展本体实例和映射规则，即可支撑后续优化模型演进。